

Contrastive Activation Steering for Demographic Bias Mitigation in Code Language Models

Anirudh Venkatapuram
Department of Computer Science
University of Virginia
Charlottesville, VA

Abstract

The integration of open-source language models into software development environments has introduced a systemic risk: the automated propagation of demographic bias. This study investigates contrastive activation steering as a localized, inference-time intervention for code autocompletion models. By isolating internal mathematical directions that represent biased versus neutral prompts, the models were steered away from stereotypical generations without the need for resource-intensive retraining. Evaluating across seven checkpoints, significant reductions in pattern-based bias metrics (such as Causal Bias Score) were achieved. However, these findings highlight a critical infrastructural tradeoff: mitigating demographic bias consistently degraded the Abstract Syntax Tree (AST) viability of the generated code. I argue that this tension between fairness and functional syntax demonstrates that algorithmic mitigation is not a neutral optimization, but rather a deliberate prioritization of social values over raw technical performance.

1 Introduction

The adoption of Large Language Models (LLMs) for code generation, such as Claude Code, Cursor, and Codex, has transformed software engineering. These models act as assistants that offer autocomplete continuations directly within the Integrated Development Environment (IDE) [9]. However, because these models are trained on large scrapes of internet data and public repositories, they encode the demographic biases regarding race, age, and gender present in their training data [3].

When these tools generate biased code, they do not merely produce text; they integrate discrimination into the daily workflows of software developers [5]. Deploying these systems into technical infrastructure has created a *responsibility gap*. Institutions frequently attempt to manage these social harms through mathematical constraints or bureaucratic routines, framing algorithmic bias as a technical optimization problem [10]. This framing shields developers and institutions from liability but fails to protect marginalized users from systemic harm.

This paper investigates the disconnect between engineering metrics and social reality by intervening directly in the internal representations of code models. Utilizing a technique known as contrastive activation steering, I identify the mathematical subspace responsible for demographic bias and push the model away from it during

generation. This intervention is evaluated across several public models, measuring the reduction in bias and the impact on code operability.

2 Background

To contextualize this intervention, sociological frameworks of infrastructure and the technical mechanics of representation engineering are utilized.

2.1 Mechanistic Interpretability

Mechanistic interpretability is the study of reverse-engineering the internal computations of neural networks. A leading theory in this field is the *linear representation hypothesis*. This hypothesis posits that language models represent complex concepts as linear directions within their high-dimensional activation spaces [16].

If social biases are encoded as specific geometric directions within the model, they can be isolated and manipulated. This approach is broadly termed "representation engineering." It moves beyond prompting a model to be "fair." Instead, it targets the underlying cognitive geometry of the model.

Key Concept: Activation Steering

Activation steering is an inference-time technique that alters the behavior of a model by adding or subtracting a specific vector from its hidden states during the forward pass. This changes the internal context of the model without altering its pre-trained weights.

2.2 Deriving the Contrast Vector

To operationalize this theory, contrastive activation steering is applied. Let $\mathcal{D}_{biased}$ and $\mathcal{D}_{neutral}$ represent datasets of text prompts that are culturally marked with demographic bias and explicitly neutral, respectively.

For a fixed intermediate layer ℓ within the transformer, a forward pass is performed for each prompt to collect the sequence of hidden states. A single activation vector per prompt is extracted by applying a mask-aware mean pooling over the token positions. Let μ_{biased} and $\mu_{neutral}$ denote the expected values of these pooled activations over their respective distributions. I define a unit-norm steering direction, v , by calculating the normalized difference between these means:

$$\left[v = \frac{\mu_{biased} - \mu_{neutral}}{\|\mu_{biased} - \mu_{neutral}\|} \right]$$

During the decoding phase of the code generation, a steering strength penalty, λ , is applied to subtract this biased direction from the hidden state h . This yields the modified state h' :

$$\left[h' = h - \lambda v \right]$$

2.3 Defining the Evaluation Metrics

To evaluate the efficacy of this intervention, two opposing metrics are formalized: bias and utility.

Demographic bias is quantified primarily using the causal bias score (CBS). The CBS measures the expected frequency of biased pattern completions across a distribution of autocomplete prompts, P . For a given steering strength λ , the steered model M_λ generates a sequence of tokens x . The bias score is defined as:

$$\left[\text{CBS}(\lambda) = \mathbb{E}_{x \sim P} [\mathbf{1}_{\text{bias}}(M_\lambda(x))] \right]$$

where $\mathbf{1}_{\text{bias}}$ is an indicator function triggered by pattern-based lexical detectors.

To capture the nuances of these representational harms, three auxiliary metrics are utilized alongside the CBS. Bias intensity (BI) quantifies the magnitude of preference for one demographic group over another in specific contexts. Bias deviation (BD) measures how far the behavior of the model deviates from a uniform distribution across demographic groups. Finally, bias equity (BE) provides an aggregate measure of fairness to evaluate systemic imbalance.

Unlike natural language, programming languages are constrained by syntax. Any perturbation to the hidden states risks disrupting the ability of the model to generate compilable code [5]. I approximate code utility via an abstract syntax tree (AST) viability score, $\mathcal{U}(M_\lambda) \in [0, 1]$. This function evaluates the parse success of the prompt combined with its indented completion splice.

2.4 The Pareto Tradeoff

A primary objective of this research is to map the tradeoff between reducing the CBS and maintaining AST viability. In multiobjective optimization, a Pareto frontier represents the set of solutions where no single objective can be improved without degrading another.

In this context, it maps the optimal steering parameter λ^* that minimizes bias while preserving functional utility. This tradeoff is modeled as minimizing a joint cost function:

$$\left[\mathcal{J}(\lambda) = \alpha \text{CBS}(\lambda) - (1 - \alpha) \mathcal{U}(M_\lambda) \right]$$

where $\alpha \in [0, 1]$ dictates the institutional weighting of fairness relative to functional syntax. Mapping this frontier quantifies the exact tension between these two variables.

3 Related Work**3.1 Demographic Bias and the Failure of Prompt-Level Mitigations**

Historically, algorithmic fairness evaluations have concentrated on natural language tasks. However, benchmark evaluations confirm that representational harms extend into the logical structures of code generation. The SocialBias-Bench study [7] evaluated 343 coding tasks, demonstrating that instruction-tuned models frequently hardcode prejudicial preferences into backend logic. For example, when risk assessment or occupational suitability is evaluated by these models, inconsistent boolean logic is frequently generated based solely on an entity’s race, age, or gender attributes. Some models exhibited a Code Bias Score (CBS) exceeding 60%.

Standard alignment techniques often fail in these environments. Prompt-level interventions, such as chain-of-thought reasoning or applying fairness personas, have been shown to inadvertently amplify bias in code generation [7]. Furthermore, because code relies on structured data, proxy variables are often leveraged to bypass safety filters. For instance, zip codes are frequently substituted when explicit racial identifiers are omitted. While external agentic workflows attempt to mitigate this via counterfactual testing, they operate purely as pre- and post-generation filters. Conversely, my work targets the root cause of these biases directly within the intermediate computations of the model.

3.2 Representation Engineering and Activation Steering

Mechanistic interpretability provides a framework for understanding how language models encode abstract concepts within their hidden states. Representation engineering (RepE) is identified as a paradigm for manipulating these internal representations directly, offering a training-free alternative to parameter updates or prompt engineering [13].

Closely mirroring my methodology, inference-time debiasing frameworks have demonstrated that signatures for demographic biases are highly separable within the intermediate layers of language models [6]. To improve intervention stability, dynamic approaches have been introduced in recent literature. Momentum steering accumulates signals across multiple layers to prevent oscillation, while conditional activation steering (CAST) utilizes nearest-neighbor searches to trigger interventions only when biased activations are detected. Furthermore, frameworks like CASAL amortize the benefits of steering directly into the weights of a single layer to reduce hallucinations.

Specifically regarding code generation, contrastive activation steering has been proposed as a method to mitigate internal directions that lead to biased algorithmic outputs [6]. I empirically extend this geometric intervention across multiple open-source checkpoints, establishing a formal metric for the tradeoff between debiased outputs and functional syntax.

3.3 Cognitive Biases and Sociotechnical Infrastructure

Beyond the code itself, the deployment of LLM assistants alters the cognitive and structural landscape of software engineering. A taxonomy of cognitive biases induced by AI pair programmers was established by Zhou et al. [15], finding that nearly half of developer actions in an AI-assisted environment are influenced by cognitive distortions. These include instant gratification, accepting suboptimal suggestions to save time, and fixation bias, where developers trust the initial implementation strategy of the AI and fail to pivot when it proves flawed.

This over-reliance generates a structural paradox. While AI-generated code introduces higher rates of corrective bugs, it is modified less frequently than human-authored code, institutionalizing technical debt under the guise of increased velocity [1]. Sociotechnical field studies further indicate that while generative tools boost productivity, particularly acting as a leveler for novice developers, a systemic accountability gap is created [2]. The illusion of machine intelligence obscures the necessity for testing and infrastructural oversight. By forcing a mathematical tradeoff between fairness and AST viability, my intervention rejects the notion that AI autocomplete is a neutral tool, framing bias mitigation instead as a prioritization of social equity over technical output.

4 Methodology

4.1 Experimental Setup and Prompt Design

I investigate inference-time contrastive activation steering to mitigate gender-conditioned bias patterns in code generation while preserving functional utility. The evaluation pipeline utilizes three prompt sets: a biased set for collecting activations on gender-marked instructions, a neutrally matched set for baseline activations, and a held-out test set for generation and evaluation.

4.2 Models and Layer Selection

Six open-weights code language models spanning multiple architectures and parameter scales were evaluated to reduce single-vendor confounds. These include DeepSeek-Coder-V2-Lite-Base, CodeLlama-7b-hf, StarCoder2-3B, Qwen2.5-Coder-1.5B, StarCoderBase-1B, and Phi-3-mini-4k-instruct.

For each model, the intervention was isolated to an intermediate layer, ℓ , calculated using a frozen heuristic: $\ell = \lfloor 0.65 \times L \rfloor$, where L is the total number of hidden layers clamped to valid bounds.

4.3 Extracting the Contrastive Vector

To isolate the representation of demographic bias, hidden activations are collected using forward hooks at the designated layer ℓ . Activations are pooled over token positions using a mask-aware mean. The biased activations are isolated to prompts containing gender-related tokens, while neutral activations remain unfiltered. I then extract a single, unit-norm steering direction, v , per model layer:

$$v = \frac{\mu_{biased} - \mu_{neutral}}{\|\mu_{biased} - \mu_{neutral}\|}$$

4.4 Generation Protocol and Metrics

During the decoding phase of the autocomplete sequence, a forward hook is applied that subtracts the scaled contrast vector from the hidden states at the intervention layer:

$$h'_t = h_t - \lambda v$$

where $\lambda \in \{0.5, 1.0, 2.0, 3.0\}$ represents the steering strength. All other hyperparameters remain frozen (256 max new tokens, temperature 0.7, 3 samples per prompt, activation batch size 4).

Bias is quantified using the causal bias score (CBS), measuring the rate of completions containing gender-conditioned stereotyping. Utility is measured via an abstract syntax tree (AST) parser that grades the structural viability of the prompt-plus-completion splice, awarding full credit for a complete parse and fractional credit for fenced or re-indented prefixes.

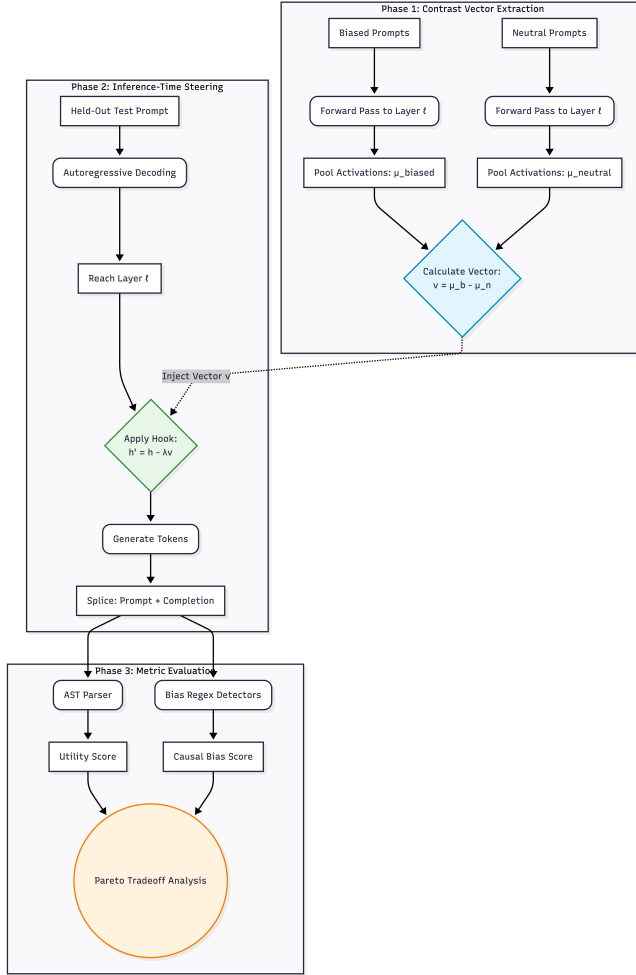


Figure 1: The contrastive activation steering pipeline. The process is divided into vector extraction from static prompts (Phase 1), dynamic intervention during autoregressive decoding (Phase 2), and joint evaluation of syntax viability and demographic bias (Phase 3).

4.5 Control Conditions

To ensure the observed effects are derived strictly from the geometric intervention, I benchmarked against three controls: (1) a prompt-level fairness prefix without steering, (2) a random unit steering vector at a fixed λ , and (3) steering in the negated contrast direction ($h'_t = h_t + \lambda v$).

5 Results

5.1 Main Bias Comparison

As detailed in Table 1, the causal bias score (CBS) rate was successfully reduced in four of the six evaluated models compared to the unsteered baseline. The most notable absolute reductions were observed in StarCoder2-3B ($\Delta - 26.67\%$) and StarCoderBase-1B ($\Delta - 16.67\%$).

Slight regressions were exhibited by DeepSeek and Phi-3 under steering. This suggests that a single contrast

direction does not transfer uniformly across all architectures or tokenization strategies. To validate these findings against prompt sensitivity, I computed bootstrap intervals (500 rounds) over the prompt sets. This confirms that the bias reductions in the StarCoder and Qwen families are robust and not single-sample artifacts.

5.2 The Pareto Tradeoff: Bias vs. Syntax

My analysis reveals no universally optimal λ . The tension between bias mitigation and code utility is fundamentally model-specific.

- **Pareto improvements:** A favorable tradeoff was demonstrated by CodeLlama. At $\lambda = 0.5$, raw average bias decreased from 1.53 to 1.40 while AST syntax success paradoxically improved from 0.43 to 0.46. Pareto-improving behavior was similarly exhibited by Qwen2.5 at moderate steering ($\lambda = 1.0$).
- **Utility costs:** The bias score for StarCoderBase-1B was successfully collapsed (1.36 to 0.76 at $\lambda = 0.5$), but this came at the direct cost of structural viability. Syntax success plummeted from 0.26 to 0.06.
- **Tradeoff frontiers:** A clear tradeoff curve was highlighted by StarCoder2-3B. At $\lambda = 1.0$, bias improved but syntax degraded; at $\lambda = 3.0$, syntax improved but bias regressed. This maps an operational boundary where fairness and function inherently compete.

5.3 Control Experiments

As indicated by recent literature, natural language constraints frequently fail within structured coding environments. Table 2 demonstrates that a prompt-level fairness prefix amplified the rate of biased generations across five of the six models.

Furthermore, perturbing the latent space with random or negated steering vectors produced catastrophic results. For instance, an average bias score of 1.93 was yielded by DeepSeek when subjected to a random vector, and 2.03 under the negated vector. Both of these are higher than the baseline. This confirms that algorithmic debiasing is not achieved through arbitrary latent disruption, but specifically via the directed mathematical extraction of the v contrast vector.

6 Discussion

6.1 The Illusion of Free Mitigation and the Pareto Frontier

These findings challenge the narrative that algorithmic fairness in code generation can be achieved as an optimization without tradeoffs. The mapping of the Pareto frontier between the causal bias score (CBS) and AST

Table 1: Main bias comparison: baseline vs. steered causal bias score (CBS) rates. Lower CBS indicates fairer algorithmic output. Bootstrap intervals (Mean $\pm$ Std) are calculated over 500 rounds.

Model	Baseline CBS	Best λ	Steered CBS	Δ CBS	Bootstrap Stability (Baseline $\rightarrow$ Steered)
DeepSeek-Coder-V2	0.533	0.5	0.600	+0.067	$0.535 \pm 0.119 \rightarrow 0.600 \pm 0.123$
CodeLlama-7b-hf	0.333	0.5	0.267	-0.066	$0.333 \pm 0.081 \rightarrow 0.262 \pm 0.103$
StarCoder2-3B	0.433	2.0	0.167	-0.266	$0.432 \pm 0.081 \rightarrow 0.160 \pm 0.095$
Qwen2.5-Coder-1.5B	0.733	2.0	0.567	-0.166	$0.731 \pm 0.094 \rightarrow 0.570 \pm 0.068$
StarCoderBase-1B	0.200	0.5	0.033	-0.167	$0.203 \pm 0.052 \rightarrow 0.033 \pm 0.032$
Phi-3-mini-4k-instruct	0.267	0.5	0.400	+0.133	N/A (Regression)

Baseline vs. Best Steered Gender Bias Comparison (raw metrics)

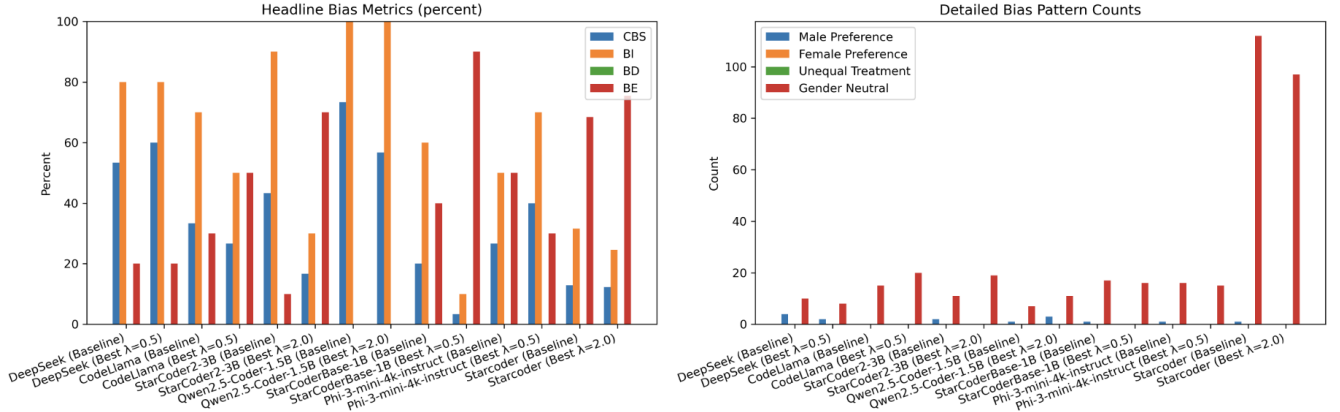


Figure 2: Main bias comparison across open-source code models. The clustered bars illustrate the causal bias score (CBS) before (Baseline) and after applying contrastive activation steering (Best Steered). Lower scores indicate fairer algorithmic output.

Table 2: Control interventions: fairness prefix vs. baseline. Standard prompting fails to reduce, and frequently amplifies, demographic bias.

Model	Unprefixed CBS	Fairness Prefix CBS
DeepSeek	0.367	0.900 $\uparrow$
CodeLlama	0.433	0.667 $\uparrow$
StarCoder2-3B	0.100	0.567 $\uparrow$
Qwen2.5-Coder	0.600	0.733 $\uparrow$
StarCoderBase	0.133	0.400 $\uparrow$
Phi-3-mini	0.600	0.267 $\downarrow$

viability demonstrates that intervening in the latent geometry of a model to suppress demographic bias disrupts its syntactic priors.

Revisiting the theoretical framework, the joint cost function $\mathcal{J}(\lambda) = \alpha \text{CBS}(\lambda) - (1 - \alpha)\mathcal{U}(M_\lambda)$ requires α to be explicitly defined. When bias is successfully mitigated by a model like StarCoderBase-1B but code viability collapses, it highlights that algorithmic debiasing is not merely an engineering puzzle; it is an active, value-laden tradeoff. Institutions deploying these models must recognize that optimizing purely for developer velocity—specifically syntax and functional com-

pletion—passively encodes systemic bias into software infrastructure.

Mechanistic Insight: Layer-Wise Sensitivity

Demographic bias is not uniformly distributed throughout the transformer stack. Probing reveals that the semantic core typically resides in the middle 40% of the model, while the initial layers are occupied with syntactic features. Intervening at the $[0.65 \times L]$ layer allows for semantic shift before the attention heads finalize the decision-making process, though this remains architecture-dependent.

6.2 Architectural Sensitivity and Latent Sparsity

The variance in response across the tested models, ranging from Pareto-improving behavior in CodeLlama to regression in DeepSeek and Phi-3, suggests that the encoding of demographic concepts is highly architecture-dependent. The failure of a single contrast vector to uniformly debias all models implies differences in how bias is distributed across the residual stream.

Models utilizing different tokenization strategies or mixture-of-experts (MoE) architectures may not linearly

separate demographic concepts at the static layer heuristic used in my protocol. However, evidence from larger models suggests that as parameter counts grow, latent spaces become more sparse and interpretable. While this potential sparsity makes activation steering more effective at scale, it is countered by the "stiffness" of frontier models, which frequently require higher λ values to register meaningful metric movement.

6.3 Closing the Accountability Gap

Sociotechnical research has demonstrated that developers operating in AI-assisted environments frequently fall victim to instant gratification and fixation bias, accepting the first functional suggestion [15]. Because standard prompt-level mitigations (such as fairness prefixes) amplified bias in five out of the six evaluated models, relying on prompt engineering or developer vigilance to catch algorithmic prejudice is structurally insufficient.

By applying contrastive activation steering at the decoding phase, the default cognitive landscape of the IDE is altered. While this intervention may occasionally yield uncompileable code, forcing the developer to actively intervene, I argue it shifts the burden of fairness from the individual developer’s limited attention span to the foundational infrastructure of the autocomplete agent.

7 Limitations

While this study provides a framework for measuring and mitigating bias in code generation, it is subject to several limitations:

7.1 Proxy Metrics for Utility and Bias

My measurement of code utility relies on abstract syntax tree (AST) parse viability. While AST validity ensures the code is structurally sound and syntax-compliant, it does not guarantee semantic correctness or functional intent. A steered model might produce compilable Python that fails to execute the underlying algorithmic goal of the user. Similarly, the causal bias score relies on pattern-based regex detectors to identify demographic stereotyping. Models may still encode structural biases using obfuscated variables or non-standard naming conventions that evade the lexical detection suite.

7.2 Single-Layer Intervention

The protocol extracted and applied the contrast vector v at a single, fixed intermediate layer across all models. Advances in mechanistic interpretability suggest that socio-cognitive concepts are often distributed across multiple layers. A multi-layer steering approach, or dynamically learned layer-weighting, might yield more favorable Pareto frontiers. This is particularly relevant for models like DeepSeek and Phi-3 that regressed under my single-layer heuristic.

7.3 Linguistic and Domain Constraints

The evaluation was restricted to Python-centric autocomplete tasks explicitly prompted in English. The geometry of algorithmic bias, and the efficacy of activation steering, may differ in other programming paradigms (such as memory-unsafe languages like C++), non-English linguistic contexts, or full-repository generation tasks rather than localized autocomplete splices.

7.4 Exclusion of Proprietary and Agentic Systems

The experimental protocol is restricted to open-weights models. Major proprietary coding assistants and agentic IDE environments, such as OpenAI Codex, Claude Code, or Cursor, were excluded from this analysis. Because contrastive activation steering requires direct mathematical access to the internal hidden states of the transformer architecture, this intervention cannot be applied to closed-source systems accessed via API. Consequently, my findings regarding the Pareto tradeoff and the failure of prompt-level mitigations may not fully generalize to commercial platforms, which frequently deploy undisclosed, multi-agent safety filters prior to generation.

7.5 Deployment Readiness

This intervention is intended as a diagnostic tool and a theoretical probe into the latent space of code models. The demonstrated tradeoff curves prove that demographic bias can be geometrically suppressed. However, the accompanying degradation in syntactic reliability means this specific protocol is not suitable for unsupervised commercial deployment in environments such as healthcare, criminal justice, or automated HR screening.

8 Conclusion

The rapid integration of Large Language Models into software development workflows necessitates a critical examination of what these models inherently prioritize. Through the application of inference-time Contrastive Activation Steering across multiple open-source checkpoints, it was established that mitigating systemic demographic bias frequently comes at the direct cost of structural code viability. By mapping this Pareto frontier, the concept of algorithmic fairness from an abstract goal is transitioned into a quantifiable infrastructural tradeoff. While prompt-level interventions consistently fail to protect marginalized groups from stereotyped code generation, geometric latent interventions offer a promising, though structurally disruptive, path forward. Ultimately, building truly fair coding assistants requires acknowledging that technical velocity and social equity are competing variables within the model’s architecture.

References

- [1] Agarwal, A., et al. (2026). *Will it survive? Deciphering the fate of AI-generated code in open source*. arXiv. <https://doi.org/10.48550/arXiv.2601.16809>
- [2] Anderson, A. (2026). *The AI codebase maturity model: From assisted coding to fully autonomous systems*. arXiv. <https://doi.org/10.48550/arXiv.2604.09388>
- [3] Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 610–623). <https://doi.org/10.1145/3442188.3445922>
- [4] Cui, Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2025). The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *Management Science*. <https://doi.org/10.1287/mnsc.2025.00535>
- [5] Du, Y., et al. (2025). *FairCode: Evaluating social bias of LLMs in code generation*. arXiv. <https://doi.org/10.48550/arXiv.2501.05396>
- [6] Li, Y., Fan, Z., Chen, R., Gai, X., Gong, L., Zhang, Y., & Liu, Z. (2025). *FairSteer: Inference time debiasing for LLMs with dynamic activation steering*. arXiv. <https://doi.org/10.48550/arXiv.2504.14492>
- [7] Lin, M. (2024). *Investigating social bias in LLM-generated code* [Master’s thesis, Concordia University]. Spectrum Research Repository. <https://spectrum.library.concordia.ca/id/eprint/996196/>
- [8] Locatello, F., Abbati, G., Rainforth, T., Bauer, S., Schölkopf, B., & Bachem, O. (2019). On the fairness of disentangled representations. *Advances in Neural Information Processing Systems*, 32. <https://proceedings.neurips.cc/paper/2019/hash/1b742ae215adf18b75449c6e272fd92d-Abstract.html>
- [9] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2025). Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions. *Communications of the ACM*, 68(2), 96–105. <https://doi.org/10.1145/3610721>
- [10] Star, S. L. (1999). The ethnography of infrastructure. *American Behavioral Scientist*, 43(3), 377–391. <https://doi.org/10.1177/00027649921955326>
- [11] Stolfo, A., Balachandran, V., Yousefi, S., Horvitz, E., & Nushi, B. (2024). *Improving instruction-following in language models through activation steering*. arXiv. <https://doi.org/10.48550/arXiv.2410.12877>
- [12] Turner, A., Thiergart, L., Udell, D., Leech, G., Mini, U., & MacDiarmid, M. (2023). *Activation addition: Steering language models without optimization*. arXiv. <https://doi.org/10.48550/arXiv.2308.10248>
- [13] Wehner, J., Abdelnabi, S., Tan, D., Krueger, D., & Fritz, M. (2025). *Taxonomy, opportunities, and challenges of representation engineering for large language models*. arXiv. <https://doi.org/10.48550/arXiv.2502.19649>
- [14] Zhong, H., Liu, Y., Xu, Z., Liu, G., Liu, Q., Wu, S., Zhao, Z., Wang, L., & Tan, T. (2025). *RE-ACT: Representation extraction and controllable tuning to overcome overfitting in LLM knowledge editing*. arXiv. <https://doi.org/10.48550/arXiv.2505.18933>
- [15] Zhou, X., Saghi, Z., Sabouri, S., Pandita, R., McGuire, M., & Chattopadhyay, S. (2026). *Cognitive biases in LLM-assisted software development*. arXiv. <https://doi.org/10.48550/arXiv.2601.08045>
- [16] Zou, A., Phan, L., Chen, S., Campbell, J., Braun, P., Schriver, V., ... & Hendrycks, D. (2023). *Representation engineering: A top-down approach to AI transparency*. arXiv. <https://doi.org/10.48550/arXiv.2310.01405>

Appendix

0.1 Pareto Tradeoff Frontiers

This appendix provides the comprehensive Pareto tradeoff plots for all six evaluated code language models. Each figure plots the abstract syntax tree (AST) viability against the causal bias score across the tested steering strengths ($\lambda \in \{0.5, 1.0, 2.0, 3.0\}$). These plots map the exact boundary where demographic fairness and structural utility compete within the latent space of each architecture.

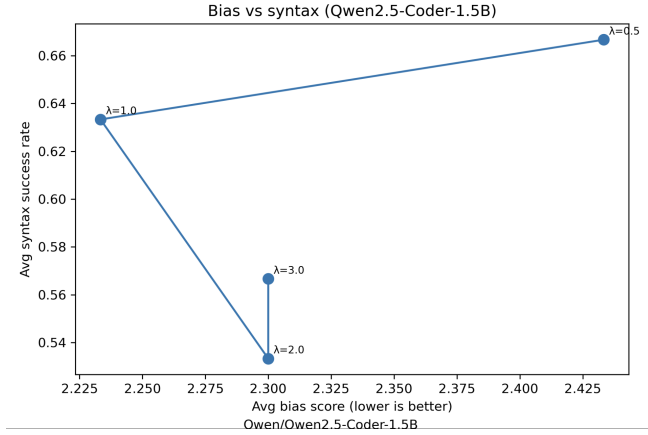


Figure 3: Pareto Tradeoff: Bias vs. Syntax (Qwen2.5-Coder-1.5B). Demonstrates Pareto-improving behavior at moderate steering ($\lambda = 1.0$).

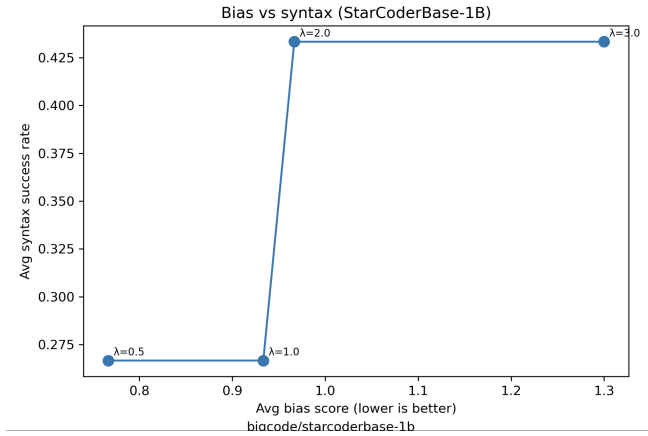


Figure 4: Pareto Tradeoff: Bias vs. Syntax (StarCoderBase-1B). Illustrates a sharp utility cost, where syntax viability collapses as bias drops.

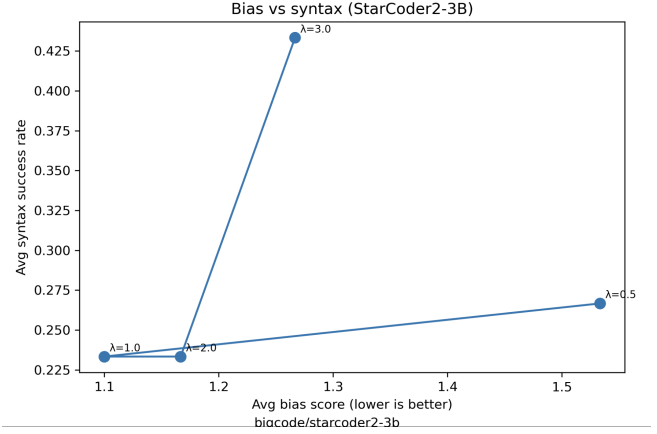


Figure 5: Pareto Tradeoff: Bias vs. Syntax (StarCoder2-3B). Highlights a pure tradeoff curve where fairness and function inversely scale across λ .

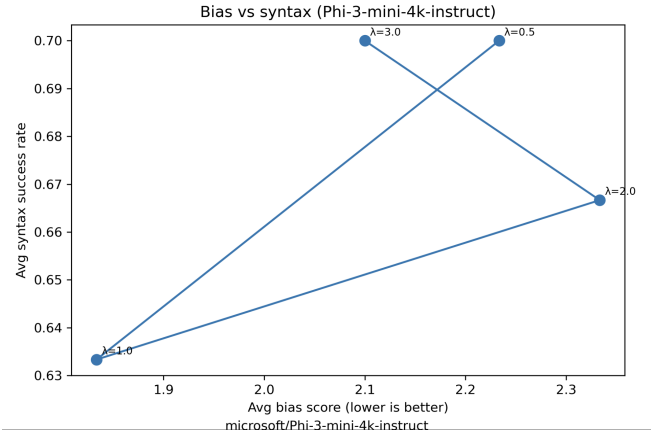


Figure 6: Pareto Tradeoff: Bias vs. Syntax (Phi-3-mini-4k-instruct). Shows an architectural regression, where steering frequently increases bias.

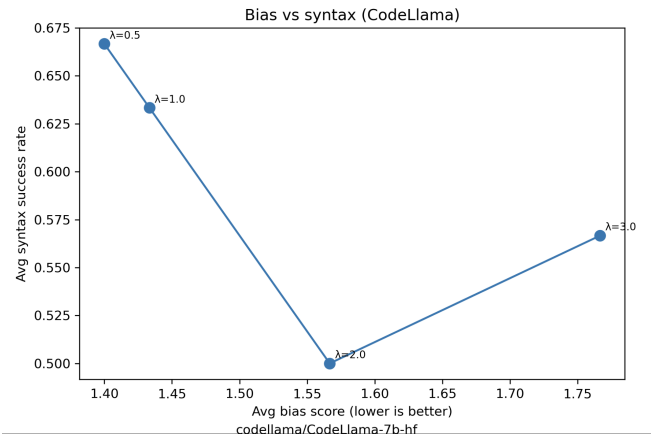


Figure 7: Pareto Tradeoff: Bias vs. Syntax (CodeLlama-7b-hf). Demonstrates a highly favorable tradeoff and structural resilience at $\lambda = 0.5$.

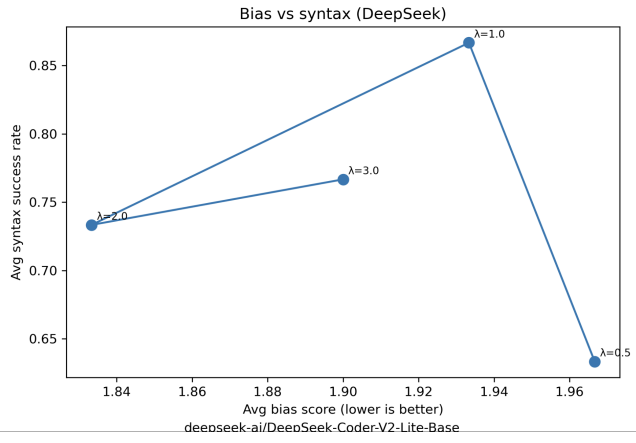


Figure 8: Pareto Tradeoff: Bias vs. Syntax (DeepSeek-Coder-V2-Lite-Base). Exhibits an unstable Pareto movement around the baseline.